Question | Help with Problem Solving | Submissions | Discussions

Your Score: 80 | Max Score: 80

## Q1. Cities starting with vowels SQL Solved

Using hints except Complete Solution is Penalty free now — Use Hint

**Problem Statement:**
Write a query to get the list of `city` names starting with vowels (i.e., a, e, i, o, or u) from station table.
The result should not contain duplicate values.

- Return the result table ordered by `city` in **ascending** order.

**Table:** station

| Column Name | Type |
|---|---|
| id | INT |
| city | VARCHAR |
| state | VARCHAR |
| nlatitude | INT |
| nlongitude | INT |

**Sample Input:**
**Table:** station

| id | city | state | nlatitude | nlongitude |
|---|---|---|---|---|
| 478 | Athens | IN | 34 | 98 |
| 619 | Arlington | CO | 75 | 93 |
| 711 | Turner | AR | 50 | 101 |
| 472 | Athens | IN | 34 | 98 |
| 839 | Slidell | LA | 85 | 152 |
| 588 | Glencoe | KY | 46 | 136 |
| 977 | Odin | IL | 53 | 116 |
| 174 | Orneway | ME | 109 | 56 |

**Sample output:**

| city |
|---|
| Arlington |
| Athens |
| Odin |
| Orneway |

**Explanation:** The above table has unique city names starting with vowels (i.e., a, e, i, o, or u) and is ordered by city in ascending order.

MySQL (MySQL 8.0)

```sql
/* YOUR QUERY GOES HERE
   Example: SELECT * FROM station;
*/

select distinct city
from station
where city like 'A%' or city like 'I%' or city like 'E%' or city like 'O%' or city like 'U%'
order by city;
```

Result

Run and See Output | Submit

## Q2. Problems that are low quality SQL Solved

Using hints except Complete Solution is Penalty free now — Use Hint

**Problem Statement:**
Write a query to report the IDs of the **low-quality** problems.
A problem is low-quality if the **like** percentage of the problem (the number of likes divided by the total number of votes) is strictly **less than 60%**.

- Return the result table ordered by `problem_id` in **ascending order**.

**Table:** problems

| Column Name | Type |
|---|---|
| problem_id | INT |
| likes | INT |
| dislikes | INT |

**Sample Input:**
**Table:** problems

| problem_id | likes | dislikes |
|---|---|---|
| 6 | 1290 | 425 |
| 11 | 2677 | 8659 |
| 1 | 4446 | 2760 |
| 7 | 8569 | 6086 |
| 13 | 2050 | 4164 |
| 10 | 9002 | 7446 |

**Sample output:**

| problem_id |
|---|
| 7 |
| 10 |
| 11 |
| 13 |

**Explanation:**
The like percentages are as follows:

- Problem 1: (4446 / (4446 + 2760)) * 100 = 61.69858%
- Problem 6: (1290 / (1290 + 425)) * 100 = 75.21866%
- Problem 7: (8569 / (8569 + 6086)) * 100 = 58.47151%
- Problem 10: (9002 / (9002 + 7446)) * 100 = 54.73006%
- Problem 11: (2677 / (2677 + 8659)) * 100 = 23.61503%
- Problem 13: (2050 / (2050 + 4164)) * 100 = 32.99002%

MySQL (MySQL 8.0)

```sql
/* YOUR QUERY GOES HERE
   Example: SELECT * FROM problems;
*/
select problem_id
from problems
where (likes/(likes+dislikes)) < 0.6
order by problem_id;
```

Result

Run and See Output | Submit

## Q3. Sort the Olympic Table SQL Solved

Using hints except Complete Solution is Penalty free now — Use Hint

**Table: Olympic**

| Column Name | Type |
|---|---|
| country | varchar |
| gold_medals | int |
| silver_medals | int |
| bronze_medals | int |

country is the primary key for this table.
Each row in this table shows a country name and the number of gold, silver, and bronze medals it won in the Olympic games.

The Olympic table is sorted according to the following rules:

- The country with more gold medals comes first.
- If there is a tie in the gold medals, the country with more silver medals comes first.
- If there is a tie in the silver medals, the country with more bronze medals comes first.
- If there is a tie in the bronze medals, the countries with the tie are sorted in ascending order lexicographically.

Write an SQL query to sort the Olympic table
The query result format is shown in the following example.

**Example :**

**Table: Olympic**

| country | gold_medals | silver_medals | bronze_medals |
|---|---|---|---|
| China | 10 | 10 | 20 |
| South Sudan | 0 | 0 | 1 |
| USA | 10 | 10 | 20 |
| Israel | 2 | 2 | 3 |
| Egypt | 2 | 2 | 2 |

MySQL (MySQL 5.6)

```sql
/* YOUR QUERY GOES HERE
   Example: SELECT * FROM EMPLOYEE;
*/
select * from Olympic
order by gold_medals desc,silver_medals desc,bronze_medals desc, country asc;
```

Result

Run and See Output | Submit

Question | Help with Problem Solving | Discussions

Your Score: 25 | Max Score: 25

## Q4. CRUD - 3 Solved

Which operation among DELETE, TRUNCATE, and DROP is **slower but allows rolling back changes**?

Choose the correct answer from below:

- DELETE
- TRUNCATE
- DROP
- None of the above

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q5. CRUD - 4 Solved

Consider the following statements regarding the **TRUNCATE operation** in SQL. Identify the **correct one**.

Choose the correct answer from below:

- It removes specified rows one-by-one from the table.
- It resets the primary key of the table.
- It can be rolled back as an intermediate step is involved.
- It is slower than DELETE.

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q6. CRUD - 5 Solved

Consider the SQL Query:

```
DROP TABLE employees;
```

What does this query do?

Choose the correct answer from below:

- Deletes all records from the 'employees' table.
- Deletes the 'employees' table structure and all its data.
- Updates the 'employees' table structure.
- Adds a new employee to the 'employees' table.

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q7. CRUD - 6 Solved

Consider the SQL Query:

```
DELETE FROM customers WHERE last_purchase_date < '2022-01-01';
```

What does this query do?

Choose the correct answer from below:

- Deletes all customers.
- Deletes customers with a last purchase date before '2022-01-01'.
- Updates the last purchase date for all customers.
- Adds a new customer with the last purchase date '2022-01-01'.

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q9. CRUD - 8 Solved

Consider the SQL query:

```
SELECT employee_name, salary FROM employees ORDER BY salary DESC LIMIT 5;
```

What is the purpose of the **LIMIT 5** in this query?

Choose the correct answer from below:

- Retrieves all employees salaries.
- Retrieves employees with the lowest 5 salaries.
- Retrieves employees with the top 5 highest salaries.
- Filters employees with salaries greater than a specific value.

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q8. CRUD - 7 Solved

Which statement is correct about the **ROLLBACK command** and the **DELETE operation** in SQL?

Choose the correct answer from below:

- DELETE can be rolled back, and it resets the primary key.
- DELETE can be rolled back, but it doesn't reset the primary key.
- DELETE cannot be rolled back, and it resets the primary key.
- DELETE cannot be rolled back, and it doesn't reset the primary key.

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit

Q10. CRUD - 9 Solved

Retrieve the names of products whose names contain the **letter 'A'** and have a price between **\$50 and \$100** (excluding 50 and 100). Choose the correct SQL query.

Choose the correct answer from below:

- SELECT product*name FROM products WHERE product*name LIKE '%A%' AND price BETWEEN 50 AND 100;
- SELECT product*name FROM products WHERE product*name LIKE 'A%' AND price BETWEEN 50 AND 100;
- SELECT product*name FROM products WHERE product*name LIKE '%A%' AND price > 50 AND price < 100;
- SELECT product*name FROM products WHERE product*name LIKE 'A%' AND price > 50 AND price < 100;

Feel Free to submit your answer. Your score will only be calculated based on the best of your 2 attempts

Submit